EECE 2311

Lab for EECE2310: Introduction to Digital Design and Computer Architecture

SPRING 2026

Lab Assignment #07

A Compact RISC-V Single-Cycle Processor

John Kimani

EECE2311 Lab

Sijing Deng

Date Submitted:

APR 07 2026

Abstract

This project will implement a compact version of the RISC-V processor with single-cycle micro-architecture utilizing modules.

1. Introduction

This lab implements a compact version of the RISC-V processor, which will implement its single-cycle micro-architecture utilizing modules from previous projects. Test this processor by executing programs of RISC-V instructions that the processor will execute the machine code generated by assembling these programs using RARS.

1.2 Simplified RISC-V Architecture

Our compact RISC-V processor can only execute four R-Type RISC-V instructions to perform the add, subtract, logic AND, and logic OR operations. The machine code format of these instructions is shown in Figure 1.

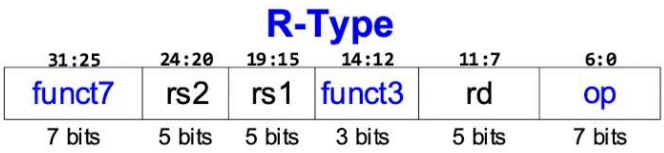


Figure 1: RISC-V R-Type Format

Figure 1: RISC-V R-Type Format

The codes of the above instructions are listed in Table 1. All of them have the same Opcode as they all are **R-Type** instructions. You can distinguish them by examining the 12th, 13th, and 30th bits of their instruction code. Both bits number 12 and 13 are part of the funct3 field, while bit number 30 is part of the funct7 field. Therefore, the control unit of this compact processor only needs to **examine these three bits** of each instruction to decide on which operation to perform.

Instruction	Description	Opcode	funct3 bits: 14-12	funct7 bits: 31-25
add rd,rs1,rs2	rd = rs1 + rs2	0110011	000	000 0000
sub rd,rs1,rs2	rd = rs1 - rs2		000	010 0000
and rd,rs1,rs2	rd = rs1 & rs2		111	000 0000
or rd,rs1,rs2	rd = rs1 | rs2		110	000 0000

Table 1: Instructions Description and Codes

ALU Control Truth Table and Boolean Expression

Bits(12th, 13th) / Bit 30th	0	1
00	ADD	SUB
01		
10	OR	
11	AND	

Table 2: Bits(12th, 13th) / Bit 30th Operations Table

Bits(13th, 12th) / Bit 30th	0	1
00	0	0
01	X	X
10	1	X
11	1	X

Table 3: Bits(12th, 13th) / Bit 30th ALU_1 Table
$ALUControl_1$ = Bit 13th

Bits(13th, 12th) / Bit 30th	0	1
00	0	1
01	X	X
10	0	X
11	1	X

Table 4: Bits(12th, 13th) / Bit 30th ALU_0 Table
$ALUControl_0$ = Bit 30th + (Bit 12th)' *Bit13th

$ALUControl_1$ = Bit 13th
$ALUControl_0$ = Bit 30th + (Bit 12th)' (Bit13th)

2. Compact RISC-V Single-Cycle Microarchitecture

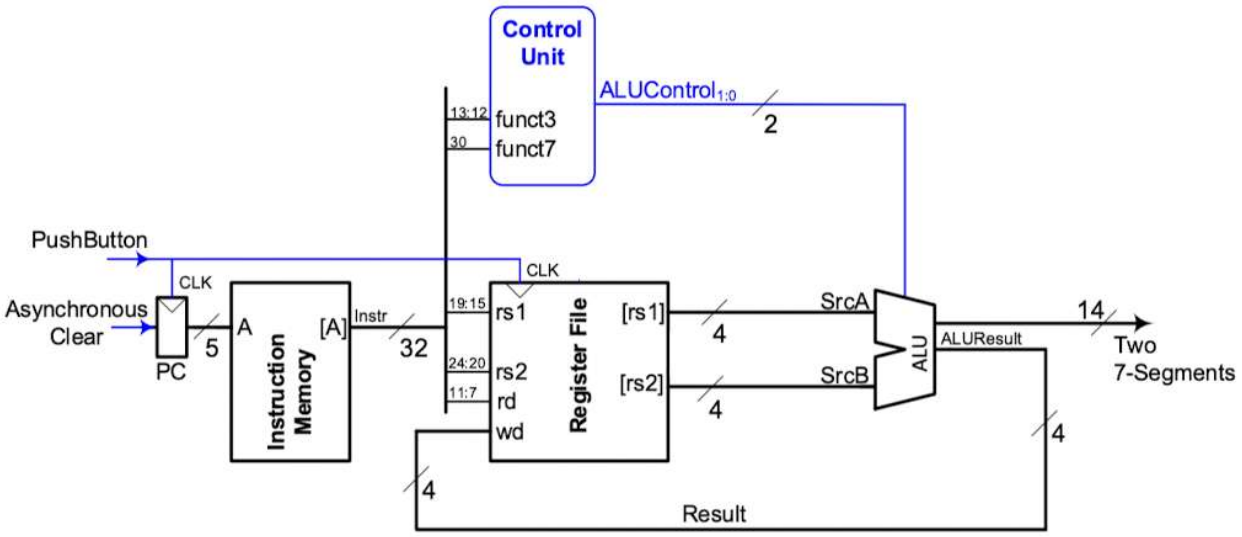


Figure 2: Microarchitecture of a Compact RISC-V Single-Cycle Processor

Figure 2 shows the block diagram of our compact RISC-V Single-Cycle microarchitecture.

Assignment 1
In reference to the microarchitecture shown in Figure 2, explain the meaning of each number or range of numbers written next to the following inputs: **funct3**, **funct7**, **rs1**, **rs2**, and **rd**.

	Meaning
funct3(13:12)	bits[13:12], decides ALUControl Type
funct7(30)	bits[30], decides ALUControl Type
rs1(19:15)	Address of the first source register
rs2(24:20)	Address of the second source register
rd(11:7)	Address of the destination register

Table 5: Bits Explanation Table

3. Microarchitecture Modules Implementation

As shown in Figure 2, the microarchitecture consists of the following four modules:

- The program counter (PC)

- The control unit.

- The instruction memory

- The register file

- The ALU

The control unit is the combinational circuits from the pre-lab assignment.

For the ALU module, you will use the one you implemented in Lab 4.

As for the PC, you need to configure it as an up only plain binary LPM_COUNTER with 5 bits output and an “asynchronous” clear input.

The implementation and interface of both the instruction memory and register file are available for download from the lab page on Canvas and they are explained in the following sub-sections.

3.1 The Instruction Memory

```
1   module InstructionMem (
2           output [31 : 0]  Instr,
3           input  [4 : 0]   A  );
4
5    //32-bit x 32 words memory
6    reg    [31:0] Instructions[31:0];
7
8    initial
9    begin
10    //Start loading instructions from the mycode.hex file to the
11    // instructions memory starting from address 0
12    $readmemh("C:/temp/mycode.hex", Instructions, 0, 30);
13                     //limit program size to 30 instructions
14
15  end
16
17  //Asynchronous reading
18  assign Instr = Instructions[A];
19
20 endmodule
```

Figure 3: Instruction memory implementation using Verilog

Line 1-3 show that the instruction memory module receives as input a 5-bit address **A**. It returns asynchronously in its **Instr** output the 32-bit content of that address, as shown in line 18. Line 6 defines the memory contents as 32 entries of 32 bits. Every time the module is created in the board, it is initialized (lines 8-15) with the content of the file mycode.hex.

3.2 The Register File

- RegisterFile.bsf

- RegisterFile.v

The .v file is the implementation written in the hardware description language (HDL) Verilog. The code is shown in Figure 4, which is explained below.

```
1  module RegisterFile (
2            output [3 : 0]  SrcA, SrcB,
3            input  [3 : 0]  wd,
4            input  [4 : 0]  rs1, rs2, rd,
5            input  clk );
6
7    //4-bit x 32 registers
8    reg [3:0] Registers[31:0];
9
10  //Asynchronous reading
11  assign SrcA = Registers[rs1];
12  assign SrcB = Registers[rs2];
13
14  //Initialize all registers
15  integer i = 0;
16  initial
17  begin
18    Registers[0] = 0; //Register x0 in RISC always has the value zero
19    for (i = 1; i < 32; i=i+1) //Do not change content of register x0
20        Registers[i] = i & 2'b11;  //store in each register a value from 0 to 3
21  end
22
23  //Synchronous writing and no writing to x0
24  always @ (posedge clk) begin
25    if ((rd != 5'b00000)) Registers[rd] = wd;
26  end
27 endmodule
```

Figure 4: Register File implementation using Verilog

Lines 1-5 show that the register file module receives five inputs. Three of them are the three 5-bit addresses, rs1, rs2, and rd. Another input is the 4-bit data input, wd, to be written to the destination address rd. The last input is the 1-bit clk input. The positive edge of clk triggers writing wd to the destination register, as shown in lines 24-26. The register file has two outputs SrcA and SrcB, . These outputs reflect the contents of the registers indicated by rs1 and rs2 respectively. Reading from the register file is done asynchronously (without the clk control) as shown in lines 11-12. Line 8 defines the register file contents as 32 registers; each has 4 bits. Every time the module is created in the board, it is initialized (lines 15-21) with the values between 0 and 3 (depending on the register number). As shown in line 18 and line 25, RISC-V register number zero must always contain value 0.

Assignment 2 Assemble the following RISC-V program in RARS and include its corresponding hexadecimal code in your report.

```
.text
main:
   add x0, x0, x0
   add x11, x3, x3
   sub x12, x1, x11
   and x13, x1, x12
   or  x14, x0, x12
```

`add x0, x0, x0`	0000000_00000_00000_000_00000_0110011
`add x11, x3, x3`	0000000_00011_00011_000_01011_0110011
`sub x12, x1, x11`	0100000_01011_00001_000_01100_0110011
`and x13, x1, x12`	0000000_01100_00001_111_01101_0110011
`or  x14, x0, x12`	0000000_01100_00000_110_01110_0110011

Table 6: Instruction to Binary Table

`add x0, x0, x0`	0x00000033
`add x11, x3, x3`	0x00318533
`sub x12, x1, x11`	0x40B08633
`and x13, x1, x12`	0x00C0F6B3
`or  x14, x0, x12`	0x00C06733

Table 7: Instruction to Hex Table

4. The Single Cycle Processor Project

Assignment 3

Use RARS to write a RISC-V assembly program of at least 10 instructions following the registers numbering as in the example given Assignment 2. Your program must include a variety of the four instructions that our compact processor executes, and it must process the contents of at least 15 different registers, including register x0.

Include the program in your report and trace it manually showing, for each instruction, the result to be stored in the destination register. Use RARS to save the Hexadecimal code of your program in file **`mycode.hex`**. Upload your design to the DE1-SOC board and verify that the result of executing each instruction matches your manual tracing.

Record a 1-minute video of the execution of the program on the board and upload it to the lab assignment on Canvas. Take a screen capture of your design and include it in your report.

- **Registers are 4-bit**

Every register starts with a value of **0, 1, 2, or 3** (cycling). Registers are **4-bit**, and they are initialized like this:

register	initial value	because..
x0	0	always 0
x1	1	1 & 0b11 = 1
x2	2	2 & 0b11 = 2
x3	3	3 & 0b11 = 3
x4	0	4 & 0b11 = 0
x5	1	5 & 0b11 = 1
x6	2	6 & 0b11 = 2
x7	3	7 & 0b11 = 3
…	repeats 0-3	…

Table 8: registers initial values

- **Inputs**

Input	Function
sw[0]	**Asynchronous Clear** resets PC back to 0, restarts the program
KEY[0]	**Clock** each press executes **one instruction**

Table 9: Inputs table

press KEY[0] once → one instruction runs → result shows on 7-seg displays.

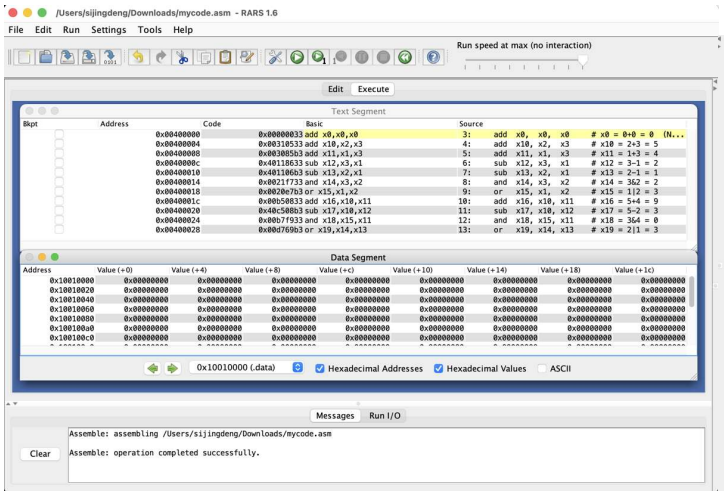


Figure 5: mycode.hex view

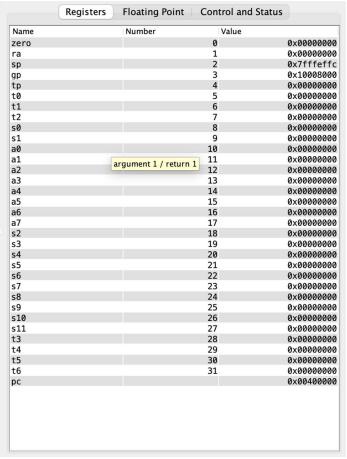


Figure 6: registers view

```
.text
main:
    add  x0,  x0,  x0     # x0 = 0+0 = 0  (NOP, safety first)
    add  x10, x2,  x3     # x10 = 2+3 = 5
    add  x11, x1,  x3     # x11 = 1+3 = 4
    sub  x12, x3,  x1     # x12 = 3-1 = 2
    sub  x13, x2,  x1     # x13 = 2-1 = 1
    and  x14, x3,  x2     # x14 = 3&2 = 2
    or   x15, x1,  x2     # x15 = 1|2 = 3
    add  x16, x10, x11    # x16 = 5+4 = 9
    sub  x17, x10, x12    # x17 = 5-2 = 3
    and  x18, x15, x11    # x18 = 3&4 = 0
    or   x19, x14, x13    # x19 = 2|1 = 3
```

Figure 7: RISC-V code view

step	instruction	calculation	result
1	`add x0, x0, x0`	0+0	x0 = 0
2	`add x10, x2, x3`	2+3	x10 = 5
3	`add x11, x1, x3`	1+3	x11 = 4
4	`sub x12, x3, x1`	3-1	x12 = 2
5	`sub x13, x2, x1`	2-1	x13 = 1
6	`and x14, x3, x2`	3&2 = 011&010	x14 = 2
7	`or x15, x1, x2`	1|2 = 001|010	x15 = 3
8	`add x16, x10, x11`	5+4	x16 = 9
9	`sub x17, x10, x12`	5+2	x17 = 3
10	`and x18, x15, x11`	3&4 = 0011&0100	x18 = 0
11	`or x19, x14, x13`	2|1 = 010|001	x19 = 3

Table 10: Manual Trace Table

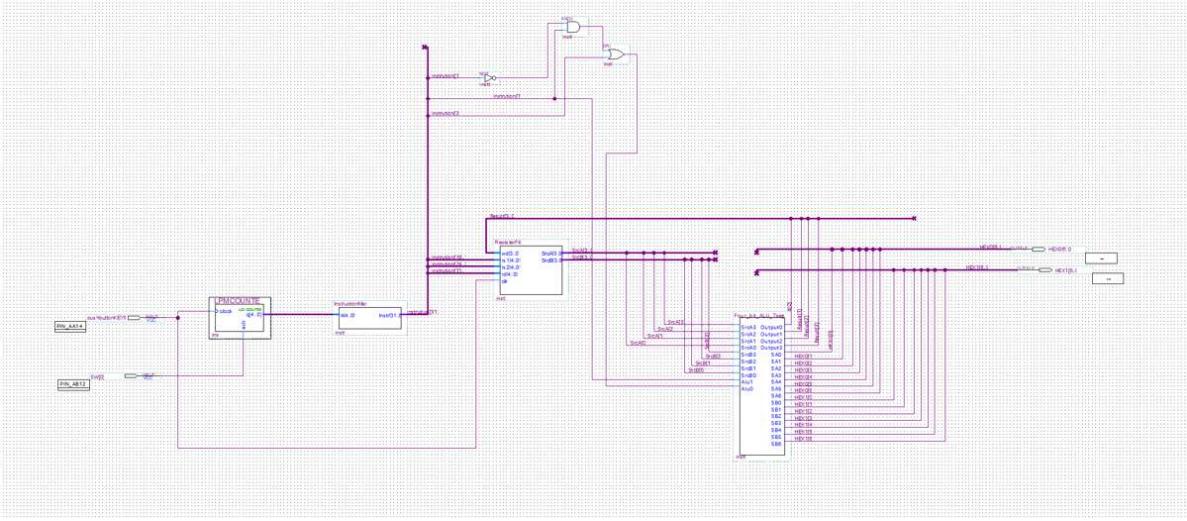


Figure 8: Single Processor Schematic

The **88** display means your circuit is getting an **undefined/floating signal.**

See Uploaded Video for board display

Analysis

In this lab, we successfully designed and implemented a compact RISC-V single-cycle processor on the DE1-SoC FPGA board capable of executing four R-Type instructions: ADD, SUB, AND, and OR. The processor was built by integrating several previously designed modules, including the ALU from Lab 4, a register file, an instruction memory, and a program counter, all connected through a schematic in Quartus.

Conclusion

The control unit was implemented as a simple combinational circuit derived from bits 12, 13, and 30 of the instruction word, which were sufficient to distinguish between all four supported instructions and generate the correct ALUControl signals. The instruction memory was loaded with machine code exported from RARS in hexadecimal format, and the program counter was driven by a push button to allow single-step execution of each instruction.

References

1. Michael Benjamin, "*Lab Report Guide*", Northeastern University, January 18, 2006.
2. Given lab 7 pdf in EECE2311 canvas.